

# TOWARDS PRACTICAL LARGE-SCALE PRIVACY-PRESERVING RECURRENT NEURAL NETWORKS

Vele Tosevski, Glenn Gulak

ECE Department, University of Toronto, Canada

## ABSTRACT

Recurrent neural networks (RNNs) are widely used in applications such as speech recognition and financial forecasting. Fully homomorphic encryption (FHE) enables computation on encrypted data, allowing third-party inference while preserving privacy. Prior work on RNN inference with FHE has been limited by computational cost. To scale and maintain efficiency, pre-activations should be represented in a single ciphertext, requiring only one programmable bootstrapping to evaluate an activation function; however, as models grow, multiply-accumulate operations overflow the message domain, degrading accuracy. Splitting pre-activations across multiple ciphertexts expands the message space to preserve accuracy but incurs exponential overhead. We propose a **training-time regularization technique that mitigates overflow by constraining pre-activations to valid regions**, enabling large-scale evaluation with single-ciphertext pre-activations. By maintaining a smaller message space, we can reduce encryption parameters, further improving efficiency. Using the CGGI FHE variant with GPU acceleration, we evaluate a 1.9M-parameter, multi-layer RNN over 28 timesteps, achieving 90.82% accuracy on encrypted MNIST with 2.1 s/sample latency—matching plaintext performance and, for the first time, demonstrating encrypted inference at this scale and efficiency.

**Index Terms**— fully homomorphic encryption, recurrent neural networks, quantization, regularization, overflow

## 1. INTRODUCTION

Machine learning drives many applications, but while data can be encrypted in transit and storage, it must be exposed during computation, creating privacy risks. Fully homomorphic encryption (FHE) [1] enables computation directly on ciphertexts. Although neural network inference under FHE has been studied [2], recurrent neural networks (RNNs) have received limited attention [3–5] due to two challenges: (i) ciphertexts accumulate noise with each operation and require *bootstrapping*, and (ii) FHE operations are computationally expensive. The temporal depth of RNNs increases operation count, making bootstrapping unavoidable. Combined with its cost, this leads to prohibitive latency even for modest models.

This work focuses on scaling RNN evaluation while maintaining performance and reducing latency. Our setting is a non-interactive client–server model: the client encrypts, the server executes FHE operations. To reduce latency, we use the CGGI [6] scheme (aka TFHE) with GPU-accelerated programmable bootstrapping (PBS), which refreshes ciphertexts while evaluating non-linear activations via lookup tables. Because CGGI supports only bounded signed integers, RNN quantization is required. Following [5], we use ternary weights and binary activations, shrinking the message space so each

pre-activation may fit in a single ciphertext, thereby reducing computation by orders of magnitude. Nevertheless, pre-activations remain vulnerable to modulus overflow, causing accuracy loss [5].

In response, we introduce **Overflow-aware Activity Regularization (OAR)**, a training-time method that constrains pre-activations to valid regions. OAR mitigates overflow by guiding pre-activations toward correct wraparound behavior. For example, with *sign* activations, an initial overflow may flip a positive to negative, but a second overflow restores correctness. Using a 1.9M-parameter multi-layer RNN, we achieve 90.82% top-1 accuracy on encrypted MNIST—only 0.17% below plaintext—with an average latency of 2.1 s/sample. This represents a  $274\times$  latency reduction over SHE [3], with negligible accuracy loss despite a  $10\times$  parameter and  $3\times$  depth increase. OAR improves accuracy by up to 71% over non-OAR baselines, with distributional analysis confirming minimal deviation between encrypted and plaintext activations.

## 2. PRELIMINARIES

### 2.1. Fully Homomorphic Encryption and the CGGI Scheme

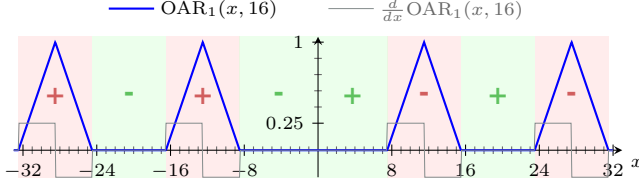
FHE, as proposed by Gentry [1], enables arithmetic on encrypted data. Its security relies on the Learning With Errors problem [7], where ciphertexts carry noise that grows with operations and eventually corrupts results. *Bootstrapping* refreshes ciphertexts by reducing noise, enabling unbounded computation [8]. Schemes are parameterized by a security level  $\lambda$ , which sets encryption parameters and key sizes; larger  $\lambda$  improves security but increases cost; we use  $\lambda \geq 128$ .

The CGGI scheme [6] operates over the real torus and in discretized form encodes integers  $x \in \mathbb{Z}_k$ , where  $k = 2^\omega$  is the plaintext modulus and  $\omega$  the bit-width. This yields a signed message space  $x \in [-k/2, k/2)$  supporting ciphertext–ciphertext, plaintext–ciphertext addition, and plaintext–ciphertext multiplication. CGGI supports programmable bootstrapping (PBS), which refreshes ciphertexts and evaluates functions via lookup tables (LUTs). Since the message space is bounded, values may overflow and wrap around (e.g.,  $7+1 = -8$  for  $k=16$ ). With *sign* activations, overflow in pre-activations can change their sign, producing incorrect outputs. If overflow is managed, correctness can be preserved. Simply enlarging parameters expands the message space but reduces efficiency. A better option is to control overflow directly, which preserves correctness and permits smaller, more efficient parameter sets without sacrificing accuracy.

### 2.2. Quantization of RNNs

Neural networks normally use floating-point arithmetic, but CGGI requires quantization into its bounded signed-integer domain [3, 9–12]. Quantization discretizes continuous values [13], introducing error that degrades performance [14]. Quantization-aware training (QAT) mitigates this by injecting noise during training [15], yet RNNs remain difficult due to recurrent dynamics that amplify errors [16]. Normalization and fixed-point quantization techniques

We acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC), RGPIN-2020-05807.



**Fig. 1:** Correct (green) and incorrect (red) regions of the  $\text{sign}$  function in  $\mathbb{Z}_{16}$ . “+” and “-” denote the result; curves show the proposed OAR penalizing incorrect regions.

have been explored [16, 17], but CGGI does not support normalization, as it reverts quantized values to floating-point, and fixed-point can only be approximated through multi-ciphertext arithmetic with PBS-heavy circuits, making RNN evaluation impractical.

Tosevski *et al.* [5] propose a four-step QAT procedure that sequentially quantizes activations, inputs, and parameters, recovering accuracy at each stage through RNN-specific optimizations. Activations are binarized to  $\{-1, 1\}$  via a single PBS evaluating the  $\text{sign}$  LUT, and parameters/inputs are ternarized to  $\{-1, 0, 1\}$ . This reduces activations to single PBS calls and additions to simple integer sums, significantly lowering latency. However, matrix multiplications still cause unbounded accumulation in pre-activations. With 11-bit precision—beyond the 8-bit guideline [18]—Tosevski *et al.* [5] reported overflow and a 25% accuracy drop. In this work, we address accumulation directly, introducing a regularization strategy that mitigates overflow and delivers major efficiency gains without compromising performance.

### 3. OVERFLOW-AWARE ACTIVITY REGULARIZATION

In a perceptron, inputs  $x \in \mathbb{R}^n$  are linearly combined with parameters  $W \in \mathbb{R}^{m \times n}$  to yield pre-activations  $z = Wx$  (biases omitted, as per the quantization strategy [5]). With ternary weights ( $W \in \{-1, 0, 1\}^{m \times n}$ ) and binary inputs ( $x \in \{-1, 1\}^n$ ),  $z$  represents integer values in  $\mathbb{Z}$ . When reduced modulo  $k = 2^\omega$ , where  $\omega$  is the bit-width, these pre-activations map to  $\mathbb{Z}_k$ , where overflow may flip the intended output of the  $\text{sign}$  activation. Figure 1 illustrates this for  $k = 16$ : green intervals preserve the correct output, while red intervals yield incorrect wraparound (e.g.,  $z = -12$  maps to “+1” instead of “-1”). Crucially, shifting a value from a red to a green region restores consistency across  $\mathbb{Z}$  and  $\mathbb{Z}_k$  (e.g., moving  $z = 12$  from  $[8, 15]$  into  $[0, 7]$  or  $[16, 23]$  recovers the correct “+1”).

This observation motivates a new activity regularizer that shifts pre-activations from incorrect to correct regions, thereby preserving consistency across  $\mathbb{Z}$  and  $\mathbb{Z}_k$ . Formally, the **Overflow-Aware Activity Regularizer (OAR)** is built from penalty functions defined for individual pre-activations  $x \in \mathbb{Z}$  under modulus  $k = 2^\omega$ :

$$\text{OAR}_1(x, k) = \text{ReLU}\left(1 - \frac{4}{k} \left| \left\lfloor \left| x + \frac{1}{2} \right| - \frac{k}{4} \right\rfloor \bmod k - \frac{k}{2} \right| \right), \quad (1)$$

$$\text{OAR}_2(x, k) = \text{OAR}_1^2(x, k). \quad (2)$$

Either form may be used:  $\text{OAR}_1$  imposes a linear penalty, while  $\text{OAR}_2$  applies a quadratic penalty that more strongly penalizes values near the center of incorrect regions. For a given layer, the OAR loss is defined as the sum of penalties across all pre-activations in that layer and is added to the global model loss during training.

As shown in Figure 1, incorrect intervals induce “hat” penalty functions whose values grow in proportion to the distance from the nearest correct region, producing a gradient that is positive on one side (directing values toward the preceding region) and negative on the other (toward the succeeding region). The gradient vanishes

---

**Algorithm 1:** Modified Four-Step Quantization Algorithm for RNNs from Tosevski *et al.* [5] (new additions in green)

---

**Input :** Vanilla RNN model  $M_0$ , trained or untrained, input dataset  $\mathcal{D}$ , ternarization scale  $t$ , **modulus**  $k = 2^\omega$  **for**  $\omega$ -bit accumulation.

**Output :** Quantized RNN with binary activations and ternary inputs/parameters  $M_4$ .

---

**Steps 1–3:** Identical to [5].

**Step 4:**

- 1 **Change activations in  $M_3$  to  $\text{ModSign}(k)$  with  $\tanh$  derivative (eq.4).**
- 2 **foreach** layer  $l$  in  $M_3$  w/ parameter distribution  $\theta_l$  **do**
- 3    $\tau_l \leftarrow t \cdot \mathbb{E}(|\theta_l|)$
- 4 **foreach** RNN layer  $l$  in  $M_3$  **do**
- 5   Set temperature scale  $s_l$  for gradient scaling.
- 6  $M_4 \leftarrow \text{train}(M_3)$  while applying (1) gradient scaling, (2) ternarization in forward step, **(3) OAR( $k$ ) in each layer.**
- 7 **if** model fails to train **then**
- 8   Increase the number of parameters in each layer of  $M_0$  by  $3\times$ , as suggested by Mishra *et al.* [19].
- 9   Repeat steps 1–3, and step 4 up to line 6.

within correct regions, preventing unnecessary updates. Unlike  $\mathcal{L}_1$  or  $\mathcal{L}_2$  regularization, which pull values unboundedly toward zero, OAR limits movement to at most  $k/4$  steps, reducing *model strain*—the magnitude of parameter adjustments required to shift pre-activations into valid regions—and thereby improving convergence stability.

We integrate OAR into **Step 4** of the four-step QAT procedure in [5] (Algorithm 1, line 6). In this step, for each layer, binarized activations are linearly transformed with ternarized parameters to produce pre-activations. OAR is then applied, adding its loss to the global objective. OAR can only mitigate overflow if it regularizes the same pre-activation distribution that arises at inference. The original procedure in [5] applies  $\text{sign}$  over  $\mathbb{Z}$ , while CGGI evaluates it over  $\mathbb{Z}_k$ , creating a mismatch: activations are drawn from  $\mathbb{Z}$  rather than their signed residues in  $\mathbb{Z}_k$ . To resolve this, we use the **ModSign activation function** in Step 4, which enforces inference-consistent activations by applying  $\text{sign}$  to signed residues in  $\mathbb{Z}_k$ :

$$\text{signed}(x, k) = \begin{cases} x \bmod k, & x \bmod k < k/2, \\ -(k - (x \bmod k)), & x \bmod k \geq k/2, \end{cases} \quad (3)$$

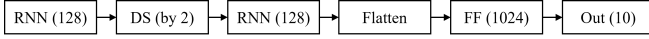
$$\text{ModSign}(x, k) = \text{sign}(\text{signed}(x, k)), \quad (4)$$

with  $\tanh$  used for backpropagation (Algorithm 1, line 1).

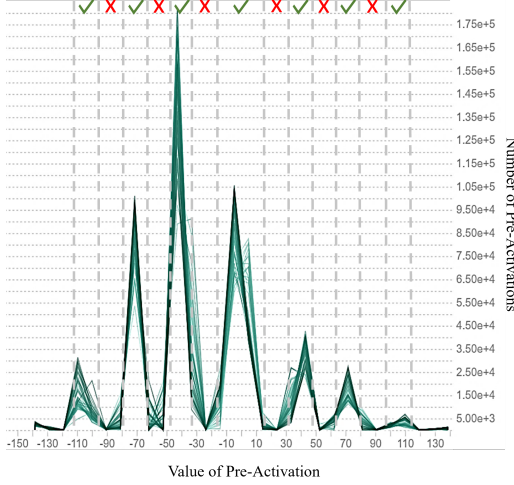
Together, OAR and ModSign ensure quantized RNNs remain accurate and efficient. ModSign aligns training with the inference-time distribution under CGGI, while OAR constrains pre-activations to valid regions of  $\mathbb{Z}_k$ , mitigating overflow. This preserves correctness within a single-ciphertext representation and enables efficient bootstrapping. By keeping residues sign-consistent under wraparound, OAR enables RNNs to scale at fixed  $\omega$ , avoiding large parameters or multi-ciphertext methods, thus addressing the limitations noted in [5]. At the same time, controlling overflow allows  $\omega$  to be varied, permitting smaller parameter sets and further efficiency gains.

### 4. EXPERIMENTAL RESULTS

Our experiments are presented in two stages. First, we evaluate OAR under plaintext simulation of the CGGI environment, where ModSign activations mimic encrypted inference. This evaluates



**Fig. 2:** Architecture of MNIST RNN. DS (by 2) refers to downsample by a factor of 2, FF refers to feed-forward. Numbers in parentheses refer to number of units. Out is the output layer.



**Fig. 3:** 5-bit pre-activation distribution of FF(1024) layer with  $OAR_2$  (rate  $10^{-4}$ ). ✓ and ✗ mark correct and incorrect regions in  $\mathbb{Z}_k$ .

$OAR$ ’s effectiveness across bit-widths, rates, and model sizes. Second, we report results under encrypted inference with CGGI, confirming that the training procedure yields effective FHE inference.

**Experimental Setup.** The MNIST RNN (Figure 2) performs digit classification on the standard 57.5k/2.5k/10k split of  $28 \times 28$  images, processing rows sequentially over 28 timesteps. The model has 1,914,368 parameters: two 128-unit RNNs (first is downsampled by 2), a flattening layer, a 1024-unit feed-forward layer, and a 10-class softmax; activations are tanh, no biases. We choose: (1) this RNN since it is over  $10\times$  larger,  $3\times$  deeper than the model from SHE [3], a widely cited baseline for RNNs under FHE, and (2) MNIST since it is a common benchmark across most FHE-ML work [9, 10, 12]. Training follows Algorithm 1: steps 1–3 use learning rate  $lr=10^{-4}$ , batch size=512, 1000 epochs each (100 for step 1); step 4 applies  $OAR_2$  with  $lr=10^{-5}$ , gradient scale  $s = 4$  for RNNs, ternarization  $t = 1.5$  (grid-searched). Adam [20], cross-entropy loss, and a cosine lr schedule decaying to  $0.1\times$  after 100 epochs are used. Data is normalized (scaled) by  $1/255$  with horizontal flips and random brightness for augmentation (training/validation only). Training is on Intel i7-7820X CPU, RTX 2080 Ti GPU, 132 GB RAM, TensorFlow v2.10, QKeras v0.9 [21]. We define an *OAR metric* as the fraction of pre-activations in correct overflow regions of  $\mathbb{Z}_k$  during inference. Encrypted inference uses CGGI [6] in a patched ConcreteCore library<sup>1</sup> (Rust v1.69) with multi-GPU support, on dual AMD EPYC 7763 CPUs, dual A100 GPUs (40GB), 512 GB RAM, Ubuntu v22.10. Threat model: honest-but-curious, inputs encrypted only, FHE  $\lambda \geq 128$ . Training and encrypted evaluation code is available.<sup>2</sup>

#### 4.1. Evaluating $OAR$ Across Bit-Widths, Rates, and Model Sizes in Large-Scale RNNs

Table 1 reports test accuracy and  $OAR$  metrics across bit-widths, with and without  $OAR_2$ . For 7- and 8-bit, accuracy differences are negligible: the pre-activation distributions are already small enough

**Table 1:** MNIST RNN test metrics across bit-widths. Columns 2 and 4 show results without  $OAR$  / with  $OAR$ . Each result underwent 1000 epochs of training with an  $OAR$  regularization rate of  $10^{-3}$ .

| Bit Width | Accuracy (%)         | Difference in Accuracy (%) | $OAR$ Metric (%) |
|-----------|----------------------|----------------------------|------------------|
| 8         | 95.30 / 95.19        | -0.11                      | 97.30 / 99.96    |
| 7         | 95.15 / 93.97        | -1.18                      | 74.72 / 99.98    |
| 6         | <b>46.82 / 89.35</b> | <b>+42.53</b>              | 48.43 / 99.96    |
| 5         | <b>9.88 / 80.73</b>  | <b>+70.85</b>              | 61.20 / 99.79    |
| 4         | 11.13 / 11.04        | -0.09                      | 52.98 / 51.79    |
| 3         | 11.14 / 11.16        | +0.02                      | 52.91 / 51.92    |

**Table 2:** MNIST RNN test accuracy with  $OAR_2$  for bit-widths of 5 or 6 across regularization rates.

| $OAR$ Rate | 5-bit Accuracy (%) | 6-bit Accuracy (%) |
|------------|--------------------|--------------------|
| $10^{-6}$  | 12.30              | 70.33              |
| $10^{-5}$  | 19.77              | 87.58              |
| $10^{-4}$  | 81.11              | 92.11              |
| $10^{-3}$  | 80.73              | 89.35              |
| $10^{-2}$  | 76.79              | 83.92              |

that integer precision alone provides adequate coverage. This is confirmed by the  $OAR$  metric, which remains consistently high even without  $OAR$ . At 5- and 6-bit, however,  $OAR$  becomes critical: accuracy improves by +71% and +43%, respectively, with the  $OAR$  metric rising from low values without  $OAR$  to nearly 100% with  $OAR$ , showing that pre-activations are successfully shifted into correct regions. Figure 3 shows the 5-bit case, where  $OAR$  visibly concentrates pre-activations into correct regions, consistent with the large gains in Table 1. Below 5-bit, both accuracy and  $OAR$  metrics collapse toward random levels, indicating that domains become too fine-grained for  $OAR$  to remain effective. To assess tunability, we vary the  $OAR_2$  regularization rate at 5- and 6-bit (Table 2). Performance for both bit-widths peak at a rate of  $10^{-4}$ , with 5-bit showing greater sensitivity to rate changes due to finer domain granularity. This shows that the  $OAR$  regularizer can be tuned like other regularization methods.

To evaluate scaling, we expanded the MNIST RNN by about  $5\times$  in parameters ( $1.9M \rightarrow 8.5M$ ) and  $5\times$  in timesteps ( $28 \rightarrow 128$ ). Using 6-bit  $OAR_2$  at rate  $10^{-4}$  with a cosine schedule initialized at  $lr=5 \cdot 10^{-6}$ , the model reached 92.27% test accuracy. This confirms that  $OAR$  enables scaling: once overflow is managed, fixed- $\omega$  evaluation remains valid even for larger RNNs. Moreover, training failed to converge without both  $OAR$  and ModSign, showing that they are jointly necessary for stable optimization.

These results show that  $OAR$  mitigates overflow and, with tuned  $\omega$  and regularization rates, enables stable training of large-scale RNNs under plaintext simulation. We now test these prepared models under encrypted inference to confirm that the simulated CGGI environment transfers to efficient and correct FHE execution.

#### 4.2. Encrypted Inference Under CGGI

We now evaluate the regular and enlarged MNIST RNNs from Section 4.1, trained with 6-bit  $OAR_2$  at a rate of  $10^{-4}$ , under encrypted inference to test our central hypothesis: overflow is the key barrier to scaling large RNNs under FHE, and once mitigated by  $OAR$ , accuracy can be preserved while efficiency improves. Fixing the message space at  $\omega = 6$  enables inference with lighter CGGI parameter sets, making large-scale RNNs practical. From [22], we adopt Set 1 (Table 4, row 5) and Set 2 (Table 8, row 12): Set 1, with larger  $N$ , yields lower noise and supports larger dot-products, while Set 2

<sup>1</sup>irskid5/concrete-core

<sup>2</sup>irskid5/overflow-aware-activity-regularization, irskid5/tfhe-rnns-mnist

**Table 3:** Performance and error metrics for regular and enlarged MNIST RNNs over encrypted data.  $\Delta$  is the accuracy difference. Error metrics include (i) average percent difference (PD) between encrypted and plaintext activations per layer, (ii) mean absolute error (MAE) of outputs.

| Model Type | Parameter Set ID | Top-1 Accuracy (%)    |          | Latency (s)     | Average PD (%)   |                  |                  | MAE               |
|------------|------------------|-----------------------|----------|-----------------|------------------|------------------|------------------|-------------------|
|            |                  | Encrypted / Plaintext | $\Delta$ |                 | 1st RNN          | 2nd RNN          | FF(1024)         |                   |
| Regular    | 1                | 90.86 / 90.99         | -0.13    | $4.86 \pm .01$  | $0.00 \pm .00$   | $0.10 \pm .14$   | $1.94 \pm 1.73$  | $2.02 \pm 1.31$   |
|            | 2                | 90.82 / 90.99         | -0.17    | $2.10 \pm .01$  | $0.56 \pm .62$   | $0.46 \pm .29$   | $5.72 \pm 1.83$  | $3.96 \pm 1.37$   |
| Enlarged   | 1                | 89.42 / 92.27         | -2.85    | $21.72 \pm .42$ | $7.85 \pm 5.25$  | $11.42 \pm 3.28$ | $34.92 \pm 1.63$ | $53.06 \pm 13.82$ |
|            | 2                | 87.56 / 92.27         | -4.71    | $10.26 \pm .03$ | $19.52 \pm 2.03$ | $20.80 \pm 1.79$ | $35.82 \pm 1.55$ | $53.38 \pm 13.92$ |

( $l_{\text{PBS}}=5$ ,  $\log_2(\beta_{\text{PBS}})=5$ ; modified) reduces  $N$  for lower latency but increases noise. Both sets provide  $\lambda \geq 128$ -bit security and support 6-bit message spaces, allowing us to test whether OAR-prepared models remain robust under noisier yet more efficient parameters. Following [5], we accumulate pre-activations in the output layer into four ciphertexts per output unit to avoid overflow. *sign* activations are evaluated using a programmable bootstrap (PBS) and accelerated on two A100 GPUs, with each streaming multiprocessor evaluating one PBS in parallel. Results are reported in Table 3, which includes top-1 accuracy, average latency, percent difference (PD) between encrypted and plaintext activations per layer, and mean absolute error (MAE) of output pre-activations, averaged over the test set. Values are reported as mean  $\pm$  one standard deviation (SD). "Encrypted" and "plaintext" denote evaluations over encrypted data and plaintext data (the latter using the 6-bit *ModSign* activation).

Using Set 1, the regular model achieves an encrypted top-1 accuracy of 90.86%, only 0.13% below plaintext. The near-zero PDs in activations and an output-layer MAE of 2.02 (far below the theoretical maximum output dot-product magnitude of 1024) confirm that overflow is effectively controlled and encrypted evaluation closely matches plaintext behavior. The average latency of 4.86 s/sample shows that even a 1.9M-parameter RNN can be run efficiently once OAR mitigates overflow. With Set 2, latency improves further to 2.10 s/sample while accuracy remains within 0.17% of plaintext; PDs and MAE rise slightly but remain small, demonstrating robustness to the additional noise from lighter parameters. Low SDs across latency and error metrics further reinforce stability. Together, these results validate our central claim: OAR-prepared models not only transfer cleanly to encrypted inference but also preserve accuracy at large scales while unlocking efficiency gains through smaller parameter sets, making practical encrypted RNN inference at scale a reality.

The regular MNIST RNN scales  $10\times$  larger and  $3\times$  deeper than prior work [3], achieving strong encrypted accuracy and over  $100\times$  greater efficiency once overflow is mitigated with OAR. The enlarged RNN pushes scale further, with about  $50\times$  more parameters,  $3\times$  greater depth, and  $5\times$  longer sequences, while still operating within the same 6-bit message space. It maintains strong accuracy under both parameter sets, demonstrating that OAR continues to enable encrypted inference without increasing  $\omega$ . While PD and MAE rise across layers, overflow remains controlled, and the model is robust even as FHE noise accumulates in larger dot-products—the higher error metrics arise from nearly  $5\times$  more multiply-add operations per layer than in the regular model. Efficiency also scales predictably, with latency increasing  $4.57\times$  in line with the longer sequences while remaining practical. Taken together, these results show that OAR enables encrypted RNNs to scale well beyond prior work, sustaining strong accuracy and efficiency at configurable  $\omega$ -bit precision.

## 5. RELATED WORK

The literature on non-interactive FHE evaluation of RNNs is limited to three key studies. (i) SHE [3] uses multiple ciphertexts per

input/activation, necessitating complex adder and activation circuits that are abundant in programmable bootstrapping operations, significantly reducing efficiency. Despite this, their fixed-point quantization approach achieves good accuracy, evaluating a single-layer RNN with 300 units over 25 timesteps on the Penn Treebank [23] dataset. This setup, with approximately 180K parameters, completes one inference in 576 seconds, with a 2.1% accuracy drop from full-precision plaintext. Our implementation runs a 1.9M-parameter, multi-layer RNN across 28 timesteps in 2.1 seconds, achieving a  $274\times$  decrease in latency for a  $10\times$  increase in parameters, maintaining over 90% accuracy with an 8 point drop from the 99% accuracy of full-precision plaintext MNIST evaluation. (ii) In another study, Podschwadt *et al.* [4] leverage batched processing and eliminate quantization by employing the CKKS [24] FHE scheme. Their method divides an RNN with  $\tau$  timesteps into  $n$  sub-RNNs, each with  $\tau/n$  timesteps. While beneficial, this approach struggles with long-term dependencies, and the significant latency of one evaluation, at 19.5 minutes, makes it unsuitable for low-latency applications. (iii) A recent study by Tosevski *et al.* [5], which forms the basis of our research, presents a 4-step quantization procedure for CGGI evaluation of RNNs that allows the use of one ciphertext per input/activation for large efficiency gains. They evaluate a 12.6M parameter RNN with attention over encrypted data across 188 timesteps, achieving a latency of 531 seconds. Despite their excellent latency for such a large model with attention, there is a -25% accuracy drop between encrypted and plaintext top-1 accuracy due to using an 11-bit plaintext modulus. Our work introduces OAR to mitigate overflow effects from using a smaller modulus, recovering this lost accuracy while retaining the efficiency benefits of their quantization technique. Successful scaling to a larger RNN suggests that [5] may benefit from using OAR. Other works [9, 10, 12] evaluate different types of neural networks over the encrypted MNIST dataset using CGGI but do not investigate RNNs.

## 6. CONCLUSION AND FUTURE WORK

In this study, we proposed methods for non-interactively evaluating large-scale RNNs over encrypted data using fully homomorphic encryption. Employing a single-ciphertext representation for RNN inputs and activations is crucial for reducing inference latency, a major obstacle to scaling RNNs, but causes reduced model performance due to numeric overflow. Introducing overflow-aware activity regularization (OAR) effectively mitigates overflow effects and restores lost accuracy, demonstrating efficacy across RNN scales. Leveraging OAR and GPU acceleration, we evaluated a 1.9M-parameter, multi-layer RNN over encrypted MNIST, achieving excellent results: 2.1 seconds latency and over 90% top-1 accuracy, establishing a new benchmark for efficient encrypted RNN inference. Future research could optimize OAR further and explore deterministic methods for handling overflow in trained networks.

## 7. REFERENCES

- [1] Craig Gentry, “Fully homomorphic encryption using ideal lattices,” in *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, New York, NY, USA, 2009, STOC ’09, p. 169–178, Association for Computing Machinery.
- [2] Robert Podschwadt, Daniel Takabi, Peizhao Hu, Mohammad H. Rafiei, and Zhipeng Cai, “A survey of deep learning architectures for privacy-preserving machine learning with fully homomorphic encryption,” *IEEE Access*, vol. 10, pp. 117477–117500, 2022.
- [3] Qian Lou and Lei Jiang, “SHE: A Fast and Accurate Deep Neural Network for Encrypted Data,” in *NeurIPS 2019*, 2019, vol. 32 of *Advances in Neural Information Processing Systems*.
- [4] Robert Podschwadt and Daniel Takabi, “Non-interactive Privacy Preserving Recurrent Neural Network Prediction with Homomorphic Encryption,” in *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*, Sept. 2021, pp. 65–70.
- [5] Vele Tosevski and Glenn Gulak, “Large-scale recurrent neural networks with fully homomorphic encryption for privacy-enhanced speaker identification,” in *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025, pp. 1–5.
- [6] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène, “TFHE: Fast Fully Homomorphic Encryption Over the Torus,” *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, Jan. 2020.
- [7] Oded Regev, “On lattices, learning with errors, random linear codes, and cryptography,” in *37th ACM STOC*, Baltimore, MD, USA, May 2005, pp. 84–93, Association for Computing Machinery.
- [8] Chiara Marcolla, Victor Sucasas, Marc Manzano, Riccardo Bassoli, Frank H. P. Fitzek, and Najwa Aaraj, “Survey on Fully Homomorphic Encryption, Theory, and Applications,” *Proceedings of the IEEE*, vol. 110, no. 10, pp. 1572–1609, 2022.
- [9] Florian Bourse, Michele Minelli, Matthias Minihold, and Pascal Paillier, “Fast Homomorphic Evaluation of Deep Discretized Neural Networks,” in *Advances in Cryptology – CRYPTO 2018*, 2018, Springer International Publishing.
- [10] Amartya Sanyal, Matt Kusner, Adria Gascon, and Varun Kanade, “TAPAS: Tricks to Accelerate (encrypted) Prediction As a Service,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, Dy Jennifer and Krause Andreas, Eds. 2018, vol. 80, pp. 4490–4499, PMLR.
- [11] Ilaria Chillotti, Marc Joye, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap, “CONCRETE: Concrete Operates on Ciphertexts Rapidly by Extending TfhE,” in *WAHC 2020–8th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, 2020, vol. 15.
- [12] Lars Folkerts, Charles Gouert, and Nektarios Georgios Tsoutsos, “REDsec: Running Encrypted Discretized Neural Networks in Seconds,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, San Diego, California, March 2023.
- [13] Tailin Liang, John Glossner, Lei Wang, Shaobo Shi, and Xiaotong Zhang, “Pruning and quantization for deep neural network acceleration: A survey,” *Neurocomputing*, vol. 461, pp. 370–403, Oct. 2021.
- [14] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer, “A Survey of Quantization Methods for Efficient Neural Network Inference,” in *Low-power computer vision : improve the efficiency of artificial intelligence*, pp. 291–326. Chapman and Hall/CRC, Boca Raton, FL, 1 edition, 2022.
- [15] Benoit Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [16] Lu Hou, Jinhua Zhu, James T. Kwok, Fei Gao, Tao Qin, and T. Liu, “Normalization Helps Training of Quantized LSTM,” in *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, pp. 7346–7356. Curran Associates Inc., 2021.
- [17] Lu Hou and James T. Kwok, “Loss-aware Weight Quantization of Deep Networks,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, Vancouver, BC, Canada, May 2018.
- [18] Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap, “Improved Programmable Bootstrapping with Larger Precision and Efficient Arithmetic Circuits for TFHE,” in *Advances in Cryptology – ASIACRYPT 2021*, Mehdi Tibouchi and Huaxiong Wang, Eds., Singapore, Singapore, 2021, pp. 670–699, Springer International Publishing.
- [19] Asit Mishra, Eriko Nurvitadhi, Jeffrey J Cook, and Debbie Marr, “WRPN: Wide Reduced-Precision Networks,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, Vancouver, BC, Canada, May 2018.
- [20] Diederik P. Kingma and Jimmy Ba, “Adam: A Method for Stochastic Optimization,” in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, Yoshua Bengio and Yann LeCun, Eds., San Diego, CA, USA, May 2015.
- [21] Claudionor N Coelho, Aki Kuusela, Shan Li, Hao Zhuang, Jennifer Ngadiuba, Thea Klæboe Aarrestad, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, and Sioni Summers, “Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors,” *Nature Machine Intelligence*, vol. 3, no. 8, pp. 675–686, Aug. 2021.
- [22] Loris Bergerat, Anas Boudi, Quentin Bourgerie, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap, “Parameter Optimization and Larger Precision for (T)FHE,” *Journal of Cryptology*, vol. 36, no. 3, pp. 28, June 2023, from <https://eprint.iacr.org/archive/2022/704/20220620:172436>.
- [23] Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz, “Building a Large Annotated Corpus of English: The Penn Treebank,” *Computational Linguistics*, vol. 19, no. 2, pp. 313–330, 1993.
- [24] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song, “Homomorphic Encryption for Arithmetic of Approximate Numbers,” in *Advances in Cryptology – ASIACRYPT 2017*, Tsuyoshi Takagi and Thomas Peyrin, Eds. 2017, pp. 409–437, Springer International Publishing.